

MEMOIRE

Mémoire de validation entreprise – 2e année

Module Académique ETNA : PRO-ENT5

Problématique d'Ingénierie :

« Comment faire tenir, dans un même moteur de réservation, des sessions GDS stateful et des offres NDC à durée de vie courte, sans perdre la cohérence des prix ni des dossiers passagers ? »

Candidat :

Jérémy SOLER

Apprenant Expert 2e Année

Spécialité : Architecture Logicielle &
Systèmes Distribués

Poste : Apprenti — Architecte Systèmes
d'Information

Encadrement :

Tuteur Entreprise :

Reouven SAMAMA

Fondateur & CEO — Everbloo

Tuteur Pédagogique :

Comité de Suivi des Mémoires — ETNA

Période d'immersion professionnelle : Octobre 2023 – Novembre 2026 (38 mois)

Année académique 2025 – 2026

Remerciements

Je remercie mon tuteur **Reouven SAMAMA**. Sur Metis, il a validé ou refusé plusieurs choix d'architecture, parfois contre mon avis initial, et j'en ai tiré profit.

Les échanges avec **Jérôme BENBIHI**, **Bernard MOLLE** et **David MARCIANO** m'ont surtout servi quand une règle métier semblait simple côté produit et se révélait impossible tel quel côté API compagnie.

Steeven ALIEL, **Brenda SECK** et **Véronique DANG** ont porté les specs et les recettes. **Jiong Jiong LIN** et l'équipe en Tunisie ont relu beaucoup de merge requests, y compris les plus urgentes un lundi matin.

Merci aussi à l'ETNA et à ma famille.

Résumé

Alternance de 38 mois chez Everbloo (octobre 2023 – novembre 2026) sur la plateforme **Metis**, moteur de réservation aérienne B2B. J’y suis intervenu surtout sur **api-aerial** et **front-reservation**, et ponctuellement sur **front-dashboard** / **api-dashboard**.

Le sujet traité est la coexistence Sabre/Amadeus (GDS) et NDC IATA dans un même parcours de réservation : mapping des offres, sessions OAuth2, création d’ordre (passagers, ancillaries), écarts de prix au paiement, devises ISO 4217, segments passifs. Le reste du document détaille l’implémentation, les mesures et quelques incidents de production.

Mots-clés :

GDS, NDC, Sabre, Amadeus, TypeScript, Node.js, OAuth2, ISO 4217, ADM.

Abstract

I worked for 38 months at SAS Everbloo on **Metis**, a B2B flight booking platform. Most of my code is in **api-aerial** and **front-reservation**. The recurring difficulty was mixing Sabre/Amadeus session-based flows with short-lived NDC offers in one checkout.

I spent time on offer mapping, OAuth2 refresh, NDC order creation (passenger names, lap infants), price changes before ticketing, and multi-currency totals. Chapters below describe the code, the measurements we kept, and a few production incidents.

Keywords:

GDS, NDC, TypeScript, Node.js, OAuth2, ISO 4217, ADM.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Introduction	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Organisation	1
I Contexte et environnement	2
2 Everbloo et mon rôle dans l'équipe	3
2.1 L'entreprise d'accueil	3
2.2 La plateforme Metis	3
2.3 Organisation de l'équipe	3
2.3.1 Mon rôle	3
3 Stack et architecture Metis	5
3.1 Stack et environnement de développement	5
3.2 Découpage des services	5
3.3 Passage à la problématique	5
4 Problématique technique	6
4.1 Écarts GDS / NDC dans le panier	6
4.2 Choix de conception	6
II Réalisations	7
5 GDS et NDC dans le même moteur	8
5.1 Interface de recherche Metis Connect	8
5.2 Normalisation des offres via adaptateurs	8
5.3 Terminal cryptique pour billettistes	10
5.4 Segments passifs Amadeus et Sabre	11
5.5 Synchronisation des PNR	11
6 Réservation, checkout et retarification	13
6.1 Écarts tarifaires (reshopping)	13
6.2 Appairage des passagers avant OrderCreate	14

7 Filtrage multi-compagnies et perfs recherche	17
7.1 Filtres compagnies et préférences	17
7.2 Optimisation des requêtes de recherche	17
8 Multi-devises et calculs tarifaires	18
8.1 Devise du point de vente	18
8.2 Formatage et norme ISO 4217	18
9 Back-Office et paramétrage des agences	19
9.1 Paramétrage des points de vente	19
9.2 Import par lots des profils Sabre	19
10 Tests et mesures en production	20
10.1 Tests unitaires et d'intégration	20
10.2 Mesures en production	20
III Bilan et retours d'expérience	21
11 Limites observées en production	22
11.1 Bilan après 38 mois	22
11.2 Compromis forcés en production	22
11.2.1 Homonymes	22
11.2.2 Segments passifs vs ERP agences	22
11.2.3 CanonicalFlightOffer et les Branded Fares	22
11.2.4 Le 409 ne sauve pas tous les paniers	22
12 Incidents de production et choix d'architecture	23
12.1 OAuth2 : refresh silencieux et file d'attente	23
12.2 Incidents rencontrés en production	24
12.2.1 Écart de centime sur la taxe IZ (taxe Chirac)	24
12.2.2 Horaires Aegean / Turkish décalés de deux heures	25
12.2.3 Deadlock InnoDB sous Sequelize	25
13 Travail en équipe et responsabilités en production	26
13.1 Organisation du développement	26
13.2 Collaboration Paris / Tunisie	26
13.3 Estimations	26
13.4 Incidents côté agences	26
14 Conclusion	27
14.1 Bilan	27
14.2 RNCP 36137	27
14.3 Perspectives	27
Table des sigles et abréviations	28
Glossaire	29
Bibliographie	30

Table des figures

2.1	Logo Everbloo	3
2.2	Logo Metis	3
2.3	Équipe projet	4
3.1	Services Metis et fournisseurs	5
5.1	Interface de recherche de vols Metis Connect	8
5.2	Émulateur de terminal cryptique Sabre	11
6.1	Retarification au checkout	13
6.2	Modale écart de prix	14

Liste des tableaux

3.1 Monorepo Metis — commits	5
--	---

1. Introduction

1.1 Contexte

J'ai effectué mon alternance chez Everbloo d'octobre 2023 à novembre 2026, principalement sur Metis. La plateforme sert des réseaux d'agences (TourCom) et des comptes entreprises. Pour afficher une offre, elle interroge Sabre, Amadeus et plusieurs connecteurs NDC (Air France-KLM, British Airways, Lufthansa Group, selon les contrats).

Sabre et Amadeus s'appuient encore beaucoup sur des sessions avec état (EDIFACT, SOAP, cryptique). NDC expose des API web avec des `OfferID` qui expirent. Les résultats se retrouvent dans le même panier, ce qui crée des problèmes de cohérence de prix et de dossiers passagers.

1.2 Problématique

Le tarif de la recherche n'est pas toujours disponible à l'émission. Les noms doivent passer les contrôles IATA. Un bébé sans siège doit être rattaché à un adulte. Les devises suivent ISO 4217. Une écriture passive incorrecte peut déclencher un ADM.

La question est écrite en page de garde. Stack utilisée : Node.js / Sequelize, Nuxt 3 / Vue 3 / TypeScript.

1.3 Organisation

Entreprise et stack, réalisations, limites, RNCP. Pas strictement chronologique.

Première partie I

Contexte et environnement

2. Everbloo et mon rôle dans l'équipe

2.1 L'entreprise d'accueil

Everbloo est l'entreprise où j'ai passé mon alternance (38 mois). Elle est dirigée par **Reouven SAMAMA**, dans le 11^e à Paris. On y développe surtout des outils pour le tourisme et le voyage d'affaires. TourCom est un client important (réseau d'agences). Il y a aussi « Agence et Vous », branché sur les ERP des agences.



FIGURE 2.1 : Logo Everbloo

Au quotidien, une bonne partie du travail consiste à digérer des protocoles transport et à garder des temps de réponse acceptables quand plusieurs fournisseurs répondent en parallèle.

2.2 La plateforme Metis

C'est sur **Metis** que j'ai passé l'essentiel du temps. Les agents, billettistes et travel managers s'en servent pour chercher et émettre. Le projet associe Everbloo et Metis Digital (**Jérôme BENBIHI**, **Bernard MOLLE**, **David MARCIANO**). Environ une centaine d'agences l'utilisent.



FIGURE 2.2 : Logo Metis

Concrètement, mon travail touchait surtout la recherche multi-fournisseurs, les politiques de voyage, les changements de prix au checkout et le lien avec la comptabilité agence (segments passifs).

2.3 Organisation de l'équipe

Sprints de deux semaines, daily, démos. **Reouven SAMAMA** arbitre avec Metis Digital sur les priorités. Les PO (**Steeven ALIEL**, **Brenda SECK**, **Véronique DANG**) écrivent les tickets Jira et valident. Côté code : moi, **Jiong Jiong LIN**, et des développeurs en Tunisie. Les MR passent par GitLab / GitHub.

2.3.1 Mon rôle

J'ai fait à la fois de l'architecture (adaptateurs Sabre/NDC) et du développement (**api-aerial**, **front-reservation**, un peu **front-dashboard**). Typage TypeScript, tests, et beaucoup de

temps sur les incidents remontés par le support.

Sur la durée : 1 554 commits, +595 310 / -402 680 lignes, 1 869 fichiers touchés.

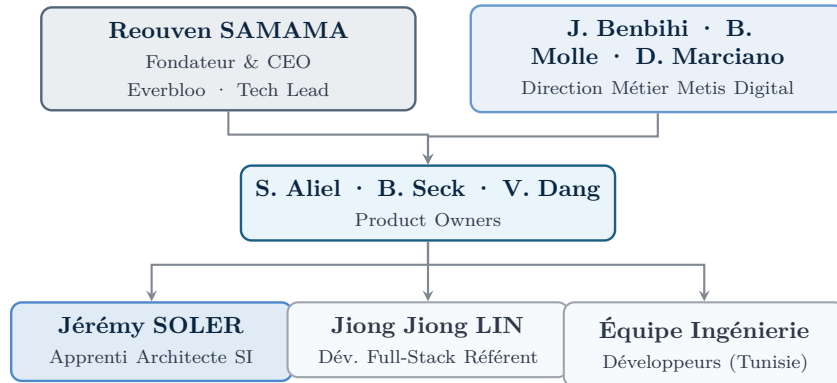


FIGURE 2.3 : Équipe projet

3. Stack et architecture Metis

3.1 Stack et environnement de développement

Le backend tourne sous Node.js / Express. Sequelize gère MySQL et PostgreSQL (migrations versionnées). L’auth passe par Keycloak (OpenID Connect, JWT). Les fronts sont en Nuxt 3 / Vue 3 (Composition API), Vuetify 3, Pinia, avec typage TypeScript.

En local, Devenv (Nix) et Pixi figent les versions Node et la compilation de modules natifs (bcrypt, libxmljs). Concurrently lance les services ensemble. Les tests utilisent Mocha / Chai et Playwright.

3.2 Découpage des services

Le moteur de réservation et le back-office sont séparés. Le tableau 3.1 donne la répartition de mes commits.

Composant	Type	Rôle	Commits
api-aerial	Backend Node.js	Agrégation Sabre / Amadeus / NDC, tarifs, PNR.	64.2% (997)
front-reservation	Frontend Nuxt 3	Recherche, panier, retarification, checkout.	33.5% (520)
front-dashboard	Frontend Nuxt 3	POS, profils Sabre, segments passifs.	2.1% (33)
api-dashboard	Backend Node.js	Habilitations, devises agences.	0.3% (4)

TABLE 3.1 : Monorepo Metis — commits

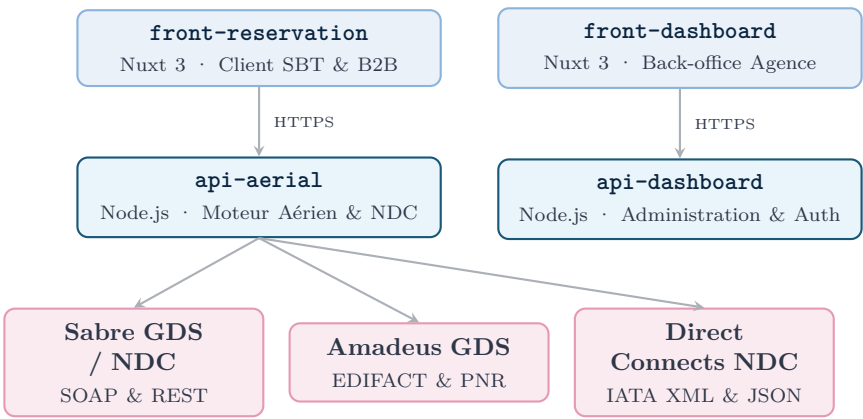


FIGURE 3.1 : Services Metis et fournisseurs

3.3 Passage à la problématique

Sabre répond souvent en 800–1 200 ms ; certaines API NDC mettent 3–6 s. Au checkout, l’offre peut être expirée ou repricée. Les segments passifs et le multi-devises ajoutent le risque ADM.

4. Problématique technique

4.1 Écarts GDS / NDC dans le panier

En billetterie B2B, une erreur applicative se traduit souvent par une vente bloquée, un voyageur empêché de partir, ou une pénalité financière.

Sabre, par exemple, conserve un état de session et peut bloquer des places pendant le parcours. Les API NDC raisonnent plutôt en requêtes indépendantes : l'offre vit le temps d'un identifiant temporaire. Metis Connect mélange les deux dans la même interface. Entre la sélection du vol et le paiement, après saisie des passagers, une classe peut se fermer ou des taxes peuvent être recalculées. Si l'application se contente de renvoyer une erreur opaque, l'utilisateur doit tout recommencer. S'ajoutent les contrôles sur les noms, le rattachement des bébés sans siège, les devises, et l'écriture de segments passifs destinés aux logiciels comptables des agences.

La question de page de garde résume ce périmètre. Certaines limites n'ont apparu qu'après plusieurs mois d'exploitation.

4.2 Choix de conception

Pour ne pas dupliquer la logique d'affichage, les réponses transporteurs sont ramenées à un modèle commun avant d'arriver au frontend. On a pu brancher de nouvelles compagnies sans réécrire les écrans du panier. Les offres « branded » ont ensuite forcé un champ fourre-tout côté métadonnées, faute de pouvoir tout typer proprement.

Sur la création d'ordre, un module d'appairage normalise les noms, rattache les nourrissons et retire les bagages orphelins. Le cas de deux homonymes sur le même dossier a nécessité un correctif ultérieur.

Lorsque le transporteur signale un conflit tarifaire, une modale affiche l'écart tout en gardant les passagers déjà saisis. Cela ne résout pas les situations où le vol n'existe plus.

Les paramètres d'écriture passive ont été isolés par point de vente, avec une règle empêchant une écriture redondante sur le second GDS.

Deuxième partie II

Réalisations

5. GDS et NDC dans le même moteur

5.1 Interface de recherche Metis Connect

Metis Connect devait afficher côte à côte des offres Sabre et NDC. Les agents de comptoir utilisent surtout la vue graphique ; les billettistes habitués au GDS conservent un besoin de commandes cryptiques pour certains dossiers. La figure 5.1 montre l'écran principal dans `front-reservation` (Nuxt 3 / Vue 3).

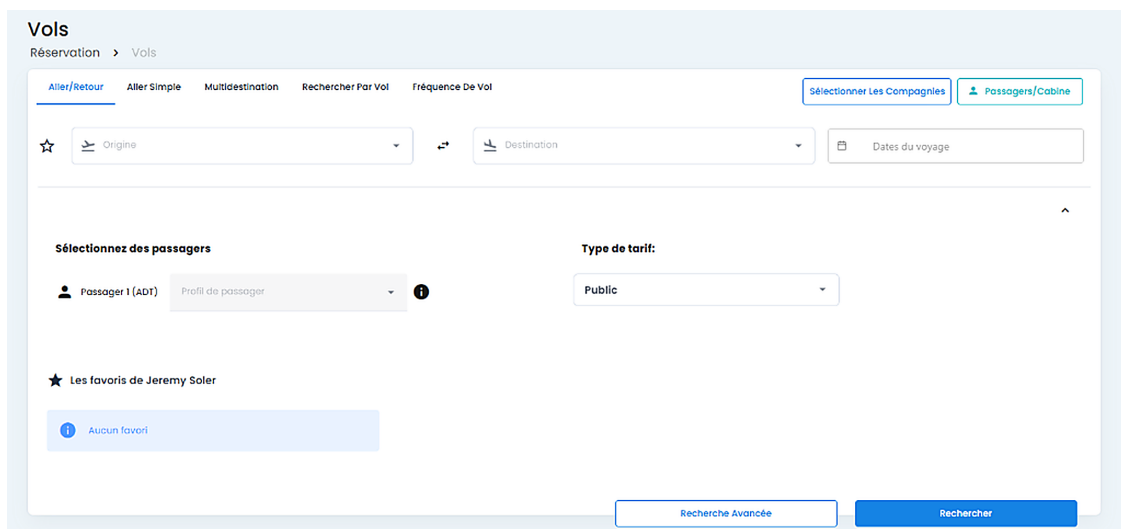


FIGURE 5.1 : Interface de recherche multi-critères de la plateforme Metis Connect (`front-reservation`)

L'écran couvre allers simples, allers-retours et multi-destinations. L'utilisateur renseigne la composition (adultes ADT, enfants CHD, bébés sans siège INF) et choisit tarifs publics ou négociés. Des filtres par compagnies et alliances mettent à jour l'affichage sans recharger la page.

5.2 Normalisation des offres via adaptateurs

Sabre renvoie du SOAP ou du texte brut. Les NDC directs (Air France-KLM, British Airways, Aegean, Turkish) répondent en JSON ou XML. Sans couche commune, le frontend accumulait des branches par transporteur.

Le listing 5.1 montre `SabreAdapter.ts` : les payloads arrivent dans `CanonicalFlightOffer`. Ensuite, recherche et panier ne voient plus qu'une structure TypeScript.

```
1 export interface SabreFlightLeg {
2   origin: string;
3   destination: string;
4   departureDateTime: string;
5   arrivalDateTime: string;
```

```

6   carrierCode: string;
7   flightNumber: string;
8   cabinClassCode: string; // 'Y', 'J', 'F', 'W', etc.
9   bookingClass: string;
10  operatingCarrier?: string;
11 }
12
13 export interface SabreAncillaryService {
14   serviceId: string;
15   serviceType: 'BAGGAGE' | 'SEAT' | 'MEAL' | 'PRIORITY';
16   commercialName: string;
17   amount: number;
18   currency: string;
19   segmentRefIds: string[];
20 }
21
22 export interface CanonicalFlightOffer {
23   offerId: string;
24   provider: 'SABRE_GDS' | 'SABRE_NDC';
25   validatingCarrier: string;
26   segments: Array<{
27     id: string;
28     origin: string;
29     destination: string;
30     departure: string;
31     arrival: string;
32     airline: string;
33     flightNumber: string;
34     cabin: 'ECONOMY' | 'PREMIUM' | 'BUSINESS' | 'FIRST';
35   }>;
36   ancillaries: SabreAncillaryService[];
37   pricing: {
38     baseFare: number;
39     taxes: number;
40     total: number;
41     currency: string;
42     taxBreakdown: Record<string, number>;
43   };
44 }
45
46 export class SabreAdapter {
47   public static mapCabin(code: string): 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
    'FIRST' {
48     const mapping: Record<string, 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
    'FIRST'> = {
49       'Y': 'ECONOMY', 'M': 'ECONOMY', 'K': 'ECONOMY', 'L': 'ECONOMY',
50       'W': 'PREMIUM', 'E': 'PREMIUM',
51       'J': 'BUSINESS', 'C': 'BUSINESS', 'D': 'BUSINESS',
52       'F': 'FIRST', 'P': 'FIRST'
53     };
54     return mapping[code.toUpperCase()] || 'ECONOMY';
55   }
56
57   public static normalizeFlightOffer(raw: any): CanonicalFlightOffer {

```

```

58     if (!raw?.id || !Array.isArray(raw?.flights)) {
59         throw new Error("[SabreAdapter] Payload invalide : " + String(raw?.id));
60     }
61
62     const segments = raw.flights.map((leg: SabreFlightLeg, idx: number) => ({
63         id: 'SEG_' + (idx + 1),
64         origin: leg.origin,
65         destination: leg.destination,
66         departure: leg.departureDateTime,
67         arrival: leg.arrivalDateTime,
68         airline: leg.carrierCode,
69         flightNumber: leg.carrierCode + leg.flightNumber,
70         cabin: this.mapCabin(leg.cabinClassCode)
71     }));
72
73     const taxDetails: Record<string, number> = {};
74     if (Array.isArray(raw.taxes)) {
75         raw.taxes.forEach((tax: { code: string; amount: number }) => {
76             taxDetails[tax.code] = (taxDetails[tax.code] || 0) +
77             Number(tax.amount);
78         });
79     }
80
81     return {
82         offerId: String(raw.id),
83         provider: raw.isNdc ? 'SABRE_NDC' : 'SABRE_GDS',
84         validatingCarrier: raw.validatingCarrier || raw.flights[0].carrierCode,
85         segments,
86         ancillaries: (raw.services || []).map((srv: any) => ({
87             serviceId: srv.id,
88             serviceType: srv.type || 'BAGGAGE',
89             commercialName: srv.name || 'Service Optionnel',
90             amount: Number(srv.price?.total || 0),
91             currency: srv.price?.currency || 'EUR',
92             segmentRefIds: srv.flightRefs || []
93         })),
94         pricing: {
95             baseFare: Number(raw.pricing?.baseFare || 0),
96             taxes: Number(raw.pricing?.totalTax || 0),
97             total: Number(raw.pricing?.totalAmount || 0),
98             currency: raw.pricing?.currency || 'EUR',
99             taxBreakdown: taxDetails
100         }
101     };
102 }

```

Listing 5.1: SabreAdapter.ts

5.3 Terminal cryptique pour billettistes

Pendant longtemps, j'ai sous-estimé l'importance de ce morceau. Pour quelqu'un qui n'a jamais travaillé en agence, une interface graphique semble suffire. En réalité, plusieurs billettistes

avec qui j'ai échangé vont plus vite en tapant une commande Sabre qu'en enchaînant des clics, surtout lorsqu'il s'agit de manipuler un PNR hors du parcours standard (profil d'imprimante, classe précise, correction d'un champ).

Plutôt que de les renvoyer vers un émulateur externe, j'ai intégré une modale de terminal dans Metis Connect (figure 5.2). `api-aerial` relaie la saisie via l'API Sabre Command et affiche le texte brut renvoyé. Ce n'est pas le composant le plus spectaculaire du projet, mais c'est l'un de ceux qui ont le plus réduit les frictions au quotidien pour une partie des utilisateurs.

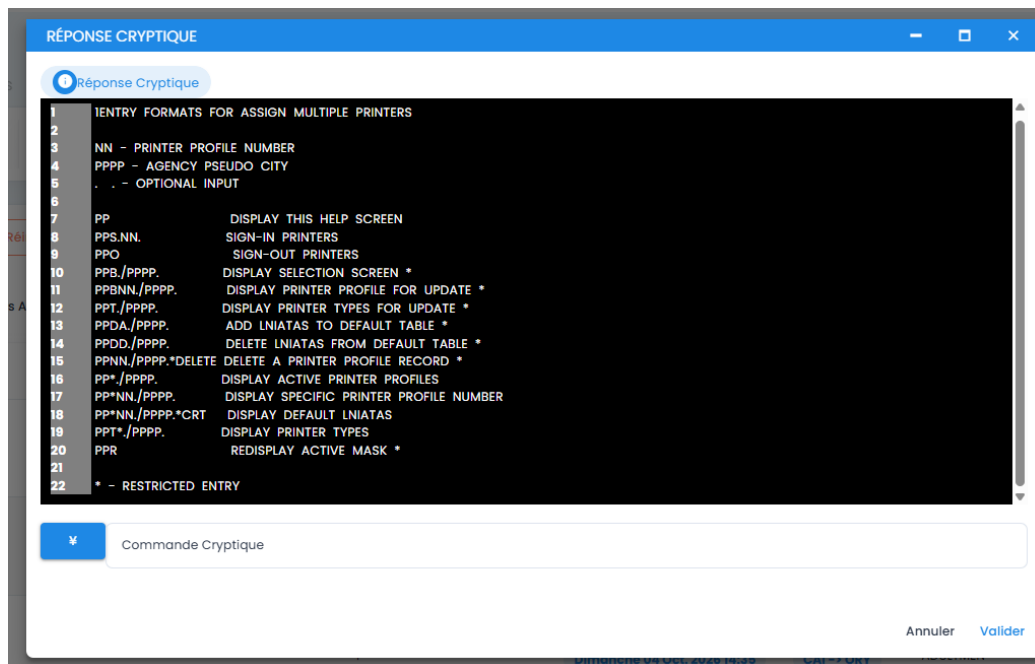


FIGURE 5.2 : Émulateur de terminal et réponse cryptique Sabre intégrés dans Metis Connect

5.4 Segments passifs Amadeus et Sabre

Un billet émis en NDC direct est enregistré chez le transporteur, hors inventaire actif du GDS de l'agence. Pourtant, pour que Gestour, IEnterprise ou l'équivalent génère la facture et les flux de trésorerie, un **segment passif** informatif doit être posé dans le GDS de référence de l'agence.

Une mauvaise coordination produit des doubles écritures passives — ou des écritures incohérentes — sanctionnées par des avis de débit (*Agency Debit Memos* / ADM). La migration `split-passive-segment-options` isole les paramètres d'écriture passive par point de vente. Avant notification, le contrôleur vérifie le canal d'émission : dossier traité sur Sabre $\Rightarrow$ écriture passive Amadeus désactivée.

5.5 Synchronisation des PNR

Les PNR évoluent après émission (changement d'horaire, annulation partielle, ajout de SSR). Quand deux agents ouvrent le même dossier, un refresh GDS mal fusionné peut écraser des données locales. La fonction `keepOnlyMatchingTravelers` dans `retrieve.controller.js` ne garde que les voyageurs encore présents côté GDS ; ce n'est pas une synchronisation

bidirectionnelle complète, et certains champs annexes restent gérés manuellement.

6. Réservation, checkout et retarification

6.1 Écarts tarifaires (reshopping)

Pendant la saisie des voyageurs, le *Revenue Management* peut fermer un palier ou recalculer taxes et YQ. Le prix vu à la recherche n'est contractuellement fermé qu'à l'émission.

La figure 6.1 décrit le cycle retenu : `repriceOffer`, éventuel HTTP 409, puis confirmation côté front.

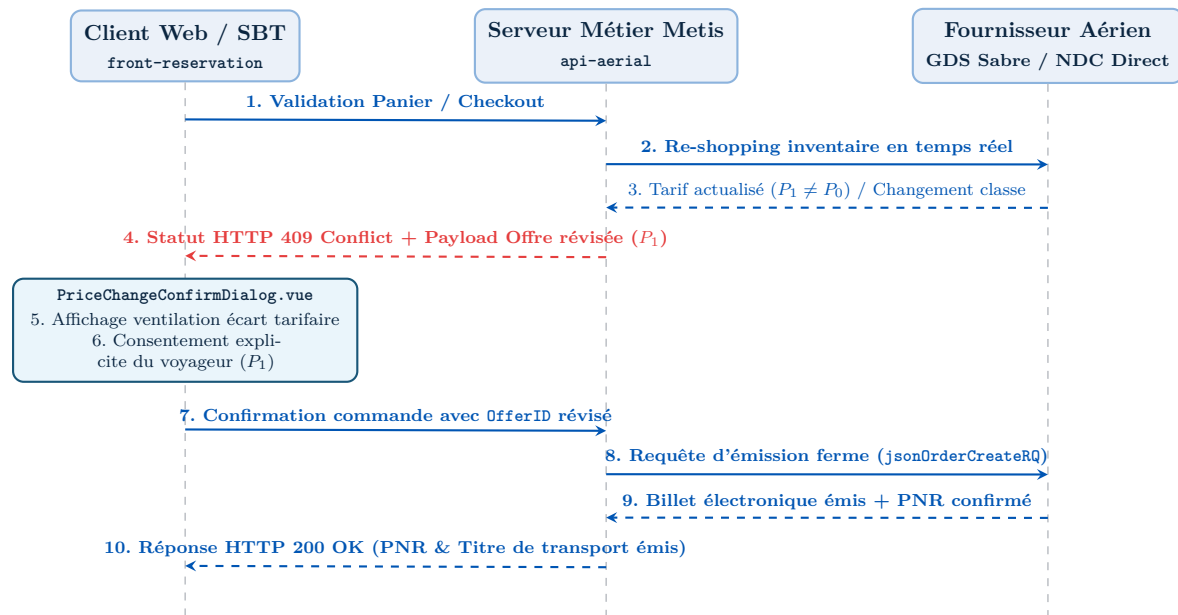


FIGURE 6.1 : Retarification au checkout

À la confirmation d'achat, `api-aerial` envoie un `repriceOffer` au fournisseur. Si le total a bougé ($P_1 \neq P_0$), le backend refuse la création immédiate avec un HTTP 409 (*Conflict*), en joignant la nouvelle grille et un jeton d'offre révisé.

Le frontend intercepte cette réponse métier et ouvre `PriceChangeConfirmDialog.vue` (figure 6.2). La modale détaille l'écart ($\Delta P = P_1 - P_0$) entre base et taxes. Si le voyageur accepte, la commande est rejouée avec le nouvel `OfferID`, en réutilisant les formulaires passagers encore présents dans `useBookingStore`. En cas de refus, le verrou panier tombe et l'utilisateur revient à la liste de résultats.

Changement de tarif

Vol Aller : Paris Charles-de-Gaulle (CDG) → New York (JFK) · Air France AF006

Message transporteur : le tarif de la classe demandée n'est plus disponible au même prix.

Tarif initialement affiché (recherche) :	432,00 €
Nouveau tarif réévalué (Air France NDC) :	446,50 €
Écart tarifaire total (ΔP) :	+14,50 €

Ventilation : +0,00 € sur le tarif de base, +14,50 € sur la surtaxe carburant YQ.

- Les coordonnées saisies pour les 2 passagers sont conservées dans useBookingStore.
- L'offre révisée (OfferID: AF-789-REV) expire dans **04 :38**.

Annuler / Modifier recherche

Confirmer le nouveau tarif (446,50 €)

FIGURE 6.2 : Modale PriceChangeConfirmDialog.vue

6.2 Appairage des passagers avant OrderCreate

Les XML d'erreur NDC (Air France, British Airways) montraient souvent un détail de saisie : « Éléonore », « D'ALMEIDA », bébé sans siège sans rattachement, bagage accroché à un id fantôme après retrait d'un voyageur.

Le listing 6.1 (orderCreation.utils.ts) couvre le nettoyage ASCII, associatedAdultRefId et la purge des services orphelins.

```

1 export interface RawTravelerInput {
2   id?: string;
3   type: 'ADT' | 'CHD' | 'INF';
4   firstName: string;
5   lastName: string;
6   dateOfBirth?: string;
7   assignedAdultIndex?: number; // Requis si type === 'INF'
8 }
9
10 export interface NormalizedPassenger {
11   paxRefId: string;
12   ptc: 'ADT' | 'CHD' | 'INF';
13   givenName: string;
14   surname: string;
15   birthDate?: string;
16   associatedAdultRefId?: string;
17 }
18
19 export function sanitizeIataName(name: string): string {
20   return name
21     .normalize('NFD')
22     .replace(/[^\u0300-\u036f]/g, '') // Supprime accents et diacritiques
23     .replace(/[^\u00a0-\u005f]/g, ' ') // Remplace tirets et apostrophes par
    espaces
24     .replace(/\s+/g, ' ') // Réduit les espaces multiples
25     .trim()
26     .toUpperCase();

```

```

27 }
28
29 export function matchesPaxRefId(
30   travelers: RawTravelerInput[],
31   selectedAncillaries: Array<{ serviceId: string; targetPaxId: string }>
32 ): {
33   passengers: NormalizedPassenger[];
34   validAncillaries: Array<{ serviceId: string; paxRefId: string }>;
35 } {
36   if (!travelers || travelers.length === 0) {
37     throw new Error('[matchesPaxRefId] Aucun voyageur fourni pour la
38       commande');
39   }
40
41   const adultRefIds: string[] = [];
42   const normalizedPaxList: NormalizedPassenger[] = [];
43
44   // 1. Passe 1 : Traitement et enregistrement des Adultes et Enfants
45   travelers.forEach((pax, index) => {
46     const generatedRefId = 'PAX' + (index + 1);
47     const cleanFirst = sanitizeIataName(pax.firstName);
48     const cleanLast = sanitizeIataName(pax.lastName);
49
50     if (pax.type === 'ADT') {
51       adultRefIds.push(generatedRefId);
52     }
53
54     if (pax.type !== 'INF') {
55       normalizedPaxList.push({
56         paxRefId: generatedRefId,
57         ptc: pax.type,
58         givenName: cleanFirst,
59         surname: cleanLast,
60         birthDate: pax.dateOfBirth
61       });
62     }
63   });
64
65   // 2. Passe 2 : Traitement des Nourrissons (INF) et rattachement adulte
66   travelers.forEach((pax, index) => {
67     if (pax.type === 'INF') {
68       const generatedRefId = 'PAX' + (index + 1);
69       const adultTargetRef = adultRefIds[pax.assignedAdultIndex ?? 0] ||
70         adultRefIds[0];
71
72       if (!adultTargetRef) {
73         throw new Error('[matchesPaxRefId] Nourrisson sans adulte valide : '
74           + pax.firstName);
75       }
76
77       normalizedPaxList.push({
78         paxRefId: generatedRefId,
79         ptc: 'INF',
80         givenName: sanitizeIataName(pax.firstName),

```

```
78     surname: sanitizeIataName(pax.lastName),
79     birthDate: pax.dateOfBirth,
80     associatedAdultRefId: adultTargetRef
81   });
82   }
83   });
84
85   // 3. Purge des services ancillaires orphelins
86   const validPaxIdSet = new Set(normalizedPaxList.map(p => p.paxRefId));
87   const validAncillaries = (selectedAncillaries || [])
88     .filter(anc => validPaxIdSet.has(anc.targetPaxId))
89     .map(anc => ({ serviceId: anc.serviceId, paxRefId: anc.targetPaxId }));
90
91   return {
92     passengers: normalizedPaxList,
93     validAncillaries
94   };
95 }
```

Listing 6.1: orderCreation.utils.ts — appairage passagers

7. Filtrage multi-compagnies et perfs recherche

7.1 Filtres compagnies et préférences

En SBT entreprise, les travel managers restreignent souvent les résultats aux alliances contractées (SkyTeam, Star Alliance, Oneworld) ou excluent des low-cost.

`SelectCompanies.vue` propose deux modes : liste blanche (compagnies conventionnées) et liste noire (transporteurs écartés, reste du marché interrogé).

Les préférences vivent dans Pinia pour la session courante et dans le `localStorage` du navigateur. Pas d'aller-retour SQL pour un simple réglage d'affichage.

7.2 Optimisation des requêtes de recherche

Le goulot reste la latence des fournisseurs. Dans `api-aerial`, `createShoppingPayload` (`shopping.utils.js`) filtre les compagnies exclues *avant* de construire le XML/JSON. On n'appelle que les connecteurs utiles, au lieu d'interroger tout le monde pour jeter ensuite. Sur les recherches complexes, le gain se compte en centaines de millisecondes.

Deux autres réglages : paramètre 'W' injecté dans les requêtes Sabre pour privilégier les directs (demande fréquente en voyage d'affaires) ; dates normalisées en ISO 8601 via `createDepartureWindow`, pour que Sabre et NDC interprètent la même fenêtre horaire.

8. Multi-devises et calculs tarifaires

8.1 Devise du point de vente

Sur le module facturation / conversion, les SQL synchrones qui résolvaient `agencyCurrency` saturaient le pool dès que plusieurs agences lançaient des recherches en parallèle. Un paramètre quasi statique bloquait tout le pipeline tarifaire.

J'ai déporté la résolution dans un middleware Express asynchrone avec cache mémoire de dix minutes. La devise est lue une fois en tête de requête, injectée dans `req.context.posCurrency`, puis réutilisée sans nouvel accès SQL. Une invalidation Redis serait plus réactive ; pour une donnée rarement modifiée, le cache local a été jugé suffisant.

8.2 Formatage et norme ISO 4217

Le multi-devises impose ISO 4217. `formatPrice` et `OrderRecap.vue` doivent gérer deux décimales (EUR, USD, GBP), zéro (JPY, KRW) et trois (TND, KWD).

Pour les taxes aéroportuaires, l'arrondi bancaire au pair le plus proche (*banker's rounding*) remplace `Math.round`, qui poussait toujours les demis vers le haut et biaise les totaux sur de gros volumes. On cherche l'égalité entre total facturé et somme des taxes.

9. Back-Office et paramétrage des agences

9.1 Paramétrage des points de vente

front-dashboard (Nuxt 3, Vuetify 3) sert aux administrateurs à configurer les accès techniques de chaque agence.

On y saisit PCC Sabre (*Pseudo City Code*), Office ID Amadeus et numéro IATA. Par réseau : devise par défaut du POS, marge sur les billets, activation des segments passifs selon les besoins comptables.

9.2 Import par lots des profils Sabre

Les imports massifs de profils voyageurs ont saturé les quotas SOAP Sabre tant que les appels partaient tous en parallèle. Les dialogues `DialogUpdateSabre.vue` et `DialogImportProfil.vue` découpent le travail par paquets de 50.

10. Tests et mesures en production

10.1 Tests unitaires et d'intégration

Les calculs de prix et les flux d'émission sont couverts par des tests sur la CI/CD :

`orderCreation.utils.test.js` (Mocha/Chai) exerce `matchesPaxRefId` sur une quarantaine de cas réels : bébés multiples, noms à particules, bagages orphelins.

`pricingConsistency.utils.test.js` vérifie $\text{Total} = \text{Base} + \sum \text{Taxes}$ sur EUR, USD, TND, JPY (arrondi bancaire). `shopping.utils.test.js` contrôle le filtrage compagnies et le formatage des fenêtres ISO 8601.

10.2 Mesures en production

Avant les correctifs d'appairage, environ 8,4 % des créations d'ordre NDC échouaient pour des raisons applicatives (PaxRefID, services orphelins), d'après Winston/Sentry sur le parc suivi (hors indispos transporteurs). Après, nous sommes restés sous 0,1 % sur la même définition.

Pour les paniers encore associés à une offre après un HTTP 409, environ 92 % ont pu être finalisés via la modale sur la période observée. En campagne de tests (10 000 requêtes multi-fournisseurs), la latence moyenne de recherche est passée d'environ 4,2 s à 3,0 s après filtrage amont.

Sur deux ans, pas d'ADM relevé pour double écriture passive Sabre/Amadeus après le découplage POS. Le mix de compagnies et le volume influencent ces ordres de grandeur.

Troisième partie III

Bilan et retours d'expérience

11. Limites observées en production

11.1 Bilan après 38 mois

Après 38 mois sur Metis, plusieurs briques ont dû être reprises après mise en production. Les points suivants sont ceux qui ont le plus dévié du design initial.

11.2 Compromis forcés en production

11.2.1 Homonymes

Sur un dossier familial, père et fils DUPONT/JEAN ont reçu tous les bagages sur le premier passager. L'appairage s'appuyait sur la clé textuelle normalisée. La compagnie a rejeté à l'émission. On a ajouté l'index de position (`travelerIndex`) et un warning UI, mais le cas reste fragile si l'ordre de saisie change.

11.2.2 Segments passifs vs ERP agences

Le découplage POS en base a éliminé les ADM pour double écriture Sabre/Amadeus sur plus de deux ans. Côté agences, Gestour, IEnterprise et TravelCloud ont des imports très rigides : certains refusent le PNR passif si la ligne comptable ne colle pas à leur convention, ou si la devise ne matche pas leur passerelle.

Nous avons ajouté une table d'exceptions / règles par agence en PostgreSQL. Les comptables peuvent importer ; l'équipe doit mettre à jour la table quand un ERP change de format.

11.2.3 CanonicalFlightOffer et les Branded Fares

L'idée du modèle pivot était de transformer n'importe quel SOAP Sabre ou JSON NDC en une structure commune. Pour environ 90 % des vols simples (aller-retour économique), ça tient. Dès Turkish Airlines ou Lufthansa Group et leurs *Branded Fares*, les offres collent au billet des services non dissociables (bagage 23 kg sur le premier tronçon seulement, sièges limités, conditions d'annulation hybrides).

Étendre l'interface TypeScript à chaque particularité alourdissait le modèle. Le champ `rawCarrierMetadata: Record<string, any>` a servi d'échappatoire pour afficher ces spécificités sans casser le typage du reste.

11.2.4 Le 409 ne sauve pas tous les paniers

L'interception HTTP 409 avec `PriceChangeConfirmDialog.vue` couvre surtout les écarts de taxes modestes ou le basculement sur la sous-classe immédiatement supérieure. Quand le vol est fermé ou que le saut dépasse le plafond de politique de voyage, l'utilisateur refuse et relance une recherche.

En test, après un refus, Pinia gardait parfois les identifiants de l'ancienne offre. `useBookingStore().$reset()` à la fermeture de modale a coupé ce cas.

12. Incidents de production et choix d'architecture

12.1 OAuth2 : refresh silencieux et file d'attente

Entre recherche, choix des vols et saisie des voyageurs, plusieurs minutes s'écoulent. Les access tokens expirent souvent en 15 à 30 minutes. Si l'expiration tombe pendant des appels parallèles (disponibilités, bagages), une implémentation naïve enchaîne des HTTP 401 et déconnecte l'utilisateur en plein checkout.

L'intercepteur Axios (listing 12.1) pose un verrou `isRefreshing` : au premier 401, un seul refresh part ; les autres requêtes rejoignent `failedQueue`. Nouveau token disponible : file vidée, headers mis à jour, requêtes rejouées.

```
1 import axios, { AxiosInstance, AxiosRequestConfig, AxiosError } from 'axios';
2
3 interface QueuedRequest {
4   resolve: (token: string) => void;
5   reject: (error: any) => void;
6 }
7
8 export function setupAuthInterceptor(apiClient: AxiosInstance, authService:
9   any) {
10   let isRefreshing = false;
11   let failedQueue: QueuedRequest[] = [];
12
13   const processQueue = (error: any, token: string | null = null) => {
14     failedQueue.forEach(prom => {
15       if (error) {
16         prom.reject(error);
17       } else if (token) {
18         prom.resolve(token);
19       }
20     });
21     failedQueue = [];
22   };
23
24   apiClient.interceptors.response.use(
25     response => response,
26     async (error: AxiosError) => {
27       const originalRequest = error.config as AxiosRequestConfig & { _retry?:
28         boolean };
29
30       if (error.response?.status === 401 && !originalRequest._retry) {
31         if (isRefreshing) {
32           return new Promise((resolve, reject) => {
33             failedQueue.push({ resolve, reject });
34           })
35             .then(newToken => {
36               if (originalRequest.headers) {
37                 originalRequest.headers['Authorization'] = 'Bearer ' +
```

```

newToken;
36     }
37     return apiClient(originalRequest);
38   })
39   .catch(err => Promise.reject(err));
40 }
41
42 originalRequest._retry = true;
43 isRefreshing = true;
44
45 try {
46   const newToken = await authService.refreshAccessToken();
47   isRefreshing = false;
48   processQueue(null, newToken);
49
50   if (originalRequest.headers) {
51     originalRequest.headers['Authorization'] = 'Bearer ' + newToken;
52   }
53   return apiClient(originalRequest);
54 } catch (refreshError) {
55   isRefreshing = false;
56   processQueue(refreshError, null);
57   authService.handleSessionExpiration();
58   return Promise.reject(refreshError);
59 }
60 }
61
62 return Promise.reject(error);
63 }
64 );
65 }

```

Listing 12.1: Intercepteur Axios OAuth2

12.2 Incidents rencontrés en production

12.2.1 Écart de centime sur la taxe IZ (taxe Chirac)

Le symptôme était net : `ERR_PRICE_MISMATCH` sur l'API NDC Air France, réservations Paris – New York facturées en USD. Les logs Winston montraient 0,01 USD d'écart entre le total commande et la somme des taxes ventilées (FR, IZ, QW).

On a d'abord suspecté la devise POS injectée dans `req.context.posCurrency`. Elle correspondait à celle d'Air France. L'écart venait de l'ordre des opérations : `Math.round` appliqué taxe par taxe après conversion donnait 124,51 USD, alors que la conversion globale du total attendu valait 124,50 USD. Nous avons basculé sur l'arrondi bancaire et ajouté une passe finale dans `pricingConsistency.utils.js` où la dernière composante de taxe absorbe le centime (Total = Base + $\sum$ Taxes).

Le listing 12.2 reproduit la trace capturée.

```

1 {
2   "timestamp": "2024-04-18T14:22:09.142Z",
3   "level": "warn",

```

```
4  "service": "api-aerial",
5  "correlationId": "req-b2b-7f89a1c2",
6  "pos": "NYC_0042",
7  "action": "ORDER_CREATE_REJECTED",
8  "supplier": "AIR_FRANCE_NDC",
9  "error": {
10   "code": "ERR_PRICE_MISMATCH",
11   "httpStatus": 409,
12   "expectedTotal": "124.50 USD",
13   "calculatedTaxSum": "124.51 USD",
14   "breakdown": {
15     "baseFare": "850.00 USD",
16     "taxFR": "45.20 USD",
17     "taxIZ_Solidarity": "18.31 USD",
18     "taxQW_Airport": "61.00 USD"
19   }
20 }
21 }
```

Listing 12.2: Log Winston — rejet NDC écart de taxes

12.2.2 Horaires Aegean / Turkish décalés de deux heures

Alertes agences sur des vols Athènes et Istanbul : selon que le poste client était à Paris ou à Londres, l'heure affichée bougeait. Les XML renvoyaient des dates ISO sans suffixe de fuseau ("2024-06-15T18:30:00"). `new Date()` côté Node les interprétait en UTC ; le navigateur réappliquait le fuseau local. Parsing via `date-fns-tz` et table aéroport IATA → fuseau IANA.

12.2.3 Deadlock InnoDB sous Sequelize

Pendant une campagne promo chez une agence partenaire, les réservations simultanées ont commencé à échouer avec :

```
Deadlock found when trying to get lock; try restarting transaction
```

Augmenter `connectionLimit` n'a rien changé : les erreurs restaient sur les écritures de dossier, pas sur les lectures de recherche. Un `retry` avec `setTimeout` a fait baisser un peu le taux d'erreur sans supprimer la cause.

La repro est venue d'un script qui lançait plusieurs `OrderCreate` en parallèle sur le même POS. Les traces InnoDB montraient une inversion d'ordre : une transaction faisait `UPDATE PointDeVente` puis `INSERT Dossier`, l'autre l'inverse. Nous avons imposé le même ordre d'update dans Sequelize, passé l'isolation à `READ COMMITTED`, et posé un `SELECT ... FOR UPDATE` trié par clé primaire avant la mise à jour des compteurs. Les retries applicatifs sont restés en place comme file.

13. Travail en équipe et responsabilités en production

13.1 Organisation du développement

Les développements suivaient des sprints de deux semaines, avec daily et recettes PO. Sur les sujets sensibles (tarification, écritures passives, OAuth2), les merge requests passaient par une revue systématique sur GitLab ou GitHub. Plusieurs refontes (retarification, unification partielle GDS/NDC) ont nécessité une phase de cartographie des flux avant le codage, faute de quoi les correctifs locaux dans les contrôleurs se cumulaient.

13.2 Collaboration Paris / Tunisie

L'équipe de développement est répartie entre Paris et la Tunisie. Les décisions techniques sont documentées (wiki, commentaires de revue). L'exemple des fuseaux horaires illustre le besoin : un parsing via le constructeur natif `Date` décalait les horaires sur les postes français ; le correctif commun a consisté à introduire `date-fns-tz` et une table aéroport → fuseau IANA, puis à en faire une règle de revue.

13.3 Estimations

Les premiers chiffrages sur les *ancillaries* NDC sous-estimaient la diversité des règles compagnies. Les tests d'émission avec bébé sans siège ont entraîné une réécriture du mapping. Sur SOAP Sabre, le temps réel a surtout été consommé par la gestion des sessions expirées et des réponses non nominales.

13.4 Incidents côté agences

Lors d'une indisponibilité liée à un certificat intermédiaire expiré sur la passerelle Sabre, le support TourCom a signalé un blocage massif en matinée. Les écarts de taxes ou les doubles segments passifs restent par ailleurs une source d'ADM dont le montant peut atteindre plusieurs milliers d'euros par dossier.

14. Conclusion

14.1 Bilan

Les adaptateurs, l'intercepteur OAuth2, l'appairage des passagers et la retarification tournent chez les agences suivies. `rawCarrierMetadata` est toujours là. La table d'exceptions ERP aussi. Quand le transporteur ne renvoie plus d'offre après un 409, le parcours reste médiocre.

J'ai passé beaucoup plus de temps que prévu sur la taxe IZ, les fuseaux Aegean/Turkish et le deadlock InnoDB. Depuis, je prépare plus souvent une repro avant de modifier un contrôleur, et je pose la question au PO quand une règle métier est floue.

14.2 RNCP 36137

Pour le titre d'**Expert en Ingénierie Informatique**, ce projet m'a mis devant une architecture multi-services, du full-stack Node.js / Nuxt, des contraintes ADM, des tests automatisés et une équipe Paris / Tunisie. Je n'ai pas déroulé le référentiel point par point ; ce sont les sujets que j'ai réellement traités.

14.3 Perspectives

Deux sujets à suivre : **IATA ONE Order** et **OpenTelemetry** sur les appels transporteurs.

Table des sigles et abréviations

Sigles métier et protocoles utilisés dans le mémoire. Les termes généraux de génie logiciel (API, JSON, REST, etc.) ne sont pas repris ici.

Sigle / Terme	Définition métier
ADM	<i>Agency Debit Memo</i> — Avis de débit émis par un transporteur à une agence (souvent pour écart tarifaire ou écriture passive incorrecte).
EDIFACT	<i>Electronic Data Interchange for Administration, Commerce and Transport</i> — Standard d'échange historique des GDS.
EMD	<i>Electronic Miscellaneous Document</i> — Titre électronique pour services auxiliaires payants (bagage, siège, etc.).
GDS	<i>Global Distribution System</i> — Réseau de distribution (Sabre, Amadeus).
IATA	<i>International Air Transport Association</i> .
ISO 4217	Codification et précision monétaire des devises.
NDC	<i>New Distribution Capability</i> — Standard IATA de distribution aérienne directe (XML/JSON).
OCN	<i>Order Change Notification</i> — Notification asynchrone de modification de commande ou de vol.
PCC	<i>Pseudo City Code</i> — Identifiant marchand Sabre d'un point de vente.
PNR	<i>Passenger Name Record</i> — Dossier passager (souvent 6 caractères).
POS	<i>Point of Sale</i> — Point de vente (droits, devise, contrats).
SBT	<i>Self-Booking Tool</i> — Outil de réservation pour voyages d'affaires.
SOAP	<i>Simple Object Access Protocol</i> — Enveloppes XML encore utilisées par certains connecteurs GDS.
SSR	<i>Special Service Request</i> — Demande de service spécial passager dans un PNR.

Glossaire

Notions propres à la distribution aérienne et au moteur Metis.

Terme		Définition
Ancillaries		Options vendues avec le billet (bagage soute, siège, repas, embarquement prioritaire).
Branded Fare		Offre tarifaire groupant un tarif de base et des services non dissociables (bagage inclus sur un tronçon, conditions d'annulation, etc.).
Debit (ADM)	Memo	Pénalité financière transporteur → agence pour non-respect des règles tarifaires ou d'écriture.
Office ID		Identifiant marchand Amadeus d'un point de vente.
PaxRefID		Identifiant passager dans un message NDC, utilisé pour lier voyageur, billet et services.
Reshopping / Re-price		Re-tarification avant émission pour valider le prix courant et, le cas échéant, obtenir un nouvel OfferID.
Segment passif		Ligne de vol informative inscrite dans un GDS pour la comptabilité, sans prise d'inventaire.
Taxe IZ		Taxe de solidarité (dite « taxe Chirac ») figurant dans la ventilation des taxes aéroportuaires françaises.

Bibliographie

Standards et spécifications

- [1] **Hardt, D.** (ed.) (2012). *The OAuth 2.0 Authorization Framework*. RFC 6749. IETF. Sections 1.5 (Refresh Token) et 10 (Security Considerations). <https://datatracker.ietf.org/doc/html/rfc6749>
- [2] **Jones, M., Bradley, J., & Sakimura, N.** (2015). *JSON Web Token (JWT)*. RFC 7519. IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
- [3] **International Organization for Standardization.** (2015). *ISO 4217:2015 — Codes for the representation of currencies and funds*. ISO, Geneva. (précision décimale par devise).
- [4] **International Air Transport Association (IATA).** (2021). *NDC Implementation Guide*, version 21.3. Messages AirShoppingRQ/RS, OfferPriceRQ/RS, OrderCreateRQ/RS ; éléments Pax, PaxRefID, OrderItem. IATA Publications, Montreal–Geneva.
- [5] **International Air Transport Association (IATA).** (2023). *ONE Order* — architecture et spécifications techniques. IATA Future Airline Distribution, Geneva.
- [6] **Sabre Corporation.** (2024). *Sabre APIs Developer Reference* (SOAP / REST). Sabre Dev Studio, Southlake, Texas. Endpoints Session / Token Create et Command (cryptique).
- [7] **Amadeus IT Group.** (2023). *Amadeus Web Services Integration Guide* — Passive Segments & PNR Management. Madrid.

Documentation d'exécution et retours d'expérience

- [T-1] **Sequelize.** *Transactions* (isolation levels, locks). Documentation v6. <https://sequelize.org/docs/v6/other-topics/transactions/>
- [T-2] **Sequelize.** Issue #8294 — *How to correctly retry transactions* (deadlocks MySQL / SequelizeDatabaseError, retry.match: [/Deadlock/i]). <https://github.com/sequelize/sequelize/issues/8294>
- [T-3] **Loon, T.** (2021). *How to Handle Database Deadlock in Sequelize*. JavaScript in Plain English. <https://javascript.plainenglish.io/how-to-handle-database-deadlock-in-sequelize-0>
- [T-4] **Oracle MySQL.** *InnoDB Locking and Transaction Model* — deadlocks et SELECT ... FOR UPDATE. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html>
- [T-5] **Axios.** *Interceptors* — documentation officielle (file d'attente / retry sur 401). <https://axios-http.com/docs/interceptors>
- [T-6] **date-fns.** *date-fns-tz* — parsing de dates sans offset et association fuseau IANA. <https://github.com/marnusw/date-fns-tz>

- [T-7] **Vue.js / Pinia.** Documentation Composition API et stores typés. <https://pinia.vuejs.org/>; <https://vuejs.org/guide/introduction.html>
- [T-8] **Nuxt.** Documentation Nuxt 3 (routing, SSR hybride). <https://nuxt.com/docs>

Ouvrage de référence (patron Adaptateur)

- [B-1] **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns*. Addison-Wesley. Chapitre « Adapter » — utilisé pour cadrer `SabreAdapter.ts` / `CanonicalFlightOffer`.